Exercices React – Niveau intermédiaire

> Ces exercices partent du principe que tu connais déjà les bases : composants fonctionnels, JSX, `props`, `useState`, listes, événements.

Exercice 1 – Compteur amélioré avec historique

Objectif: manipuler `useState`, les événements et les listes.

- Crée un composant `AdvancedCounter`.
- Affiche un nombre `count` initialisé à `0`.
- Ajoute trois boutons : `+1`, `-1`, `Reset`.
- À chaque clic (sur n'importe lequel des boutons), enregistre la nouvelle valeur de `count` dans un tableau `history`.
- Affiche sous le compteur la liste de toutes les valeurs successives (ex: `Historique : 0 → 1 → 2 → 1 → 0`).

Bonus

- Affiche aussi le nombre total de clics effectués.
- Ajoute un bouton `Annuler dernier` qui revient à la valeur précédente en s'appuyant sur `history`.

Exercice 2 – Liste de tâches avec filtres

Objectif: gérer une liste d'objets, des formulaires contrôlés et des filtres.

- Crée un composant `TodoList`.
- Utilise un `useState` pour stocker un tableau de tâches, par exemple :
  - `{ id: 1, title: 'Apprendre React', done: false }`
- Ajoute un formulaire avec un champ texte et un bouton `Ajouter`.
  - Quand tu soumets, ajoute une nouvelle tâche avec `done: false` et un `id` unique.
- Affiche la liste des tâches avec :
  - une checkbox pour marquer la tâche comme faite / non faite,
  - un bouton `Supprimer`.

Filtres

- Ajoute trois boutons : `Toutes`, `Actives`, `Terminées`.

- Utilise un `useState` pour stocker le filtre courant.
- Affiche les tâches en fonction du filtre sélectionné.

Exercice 3 – Carte utilisateur avec édition en ligne

Objectif: travailler avec plusieurs états liés à un même objet.

- Crée un composant `UserCard` qui affiche :
  - `nom`
  - `email`
  - `age`
- Initialise les valeurs avec un objet dans le `useState`, par exemple :
  - `{ name: 'Alice', email: 'alice@example.com', age: 25 }`
- Affiche ces informations dans une “carte” (un bloc avec un peu de style).

Mode édition

- Ajoute un bouton `Modifier`.
- Quand on clique sur `Modifier` :
  - remplace les textes par des champs de formulaire contrôlés (`input`),
  - le bouton `Modifier` devient `Sauvegarder`.
- Au clic sur `Sauvegarder` :
  - mets à jour l’objet dans le `useState`,
  - repasse en mode “affichage” (plus de champs input).

Bonus

- Ajoute un bouton `Annuler` en mode édition pour revenir aux anciennes valeurs sans les enregistrer.

Exercice 4 – Galerie d’images avec sélection

Objectif: manipuler les `props` et un état de “sélection”.

- Crée un composant parent `Gallery`.
- Dans ce composant, définis un tableau d’images, par exemple :
  - `{ id: 1, title: 'Chat', url: 'https://...' }`, etc.
- Crée un composant enfant `GalleryItem` qui reçoit les `props` :
  - `title`
  - `url`

    - `isSelected`
    - `onSelect` (fonction)
- Affiche la liste des `GalleryItem` dans `Gallery`.

Sélection

- Dans `Gallery`, crée un `useState` pour l'`id` de l'image sélectionnée (ou `null`).
- Quand on clique sur une image (ou un bouton dedans), appelle `onSelect(id)` pour mettre à jour l'état dans le parent.
- L'élément sélectionné doit visuellement être différent (bordure, fond, etc.).

Bonus

- En haut de la galerie, affiche le `title` de l'image sélectionnée.
- Si aucune image n'est sélectionnée, affiche "Aucune image sélectionnée".

Exercice 5 – Chronomètre (start / stop / reset)

Objectif: introduire `useEffect` avec un intervalle simple.

- Crée un composant `Timer`.
- Crée un état `seconds` initialisé à `0`.
- Crée un état `isRunning` (booléen).
- Ajoute trois boutons :
    - `Démarrer` (met `isRunning` à `true`)
    - `Arrêter` (met `isRunning` à `false`)
    - `Réinitialiser` (met `seconds` à `0` et `isRunning` à `false`)

useEffect

- Utilise `useEffect` pour démarrer un `setInterval` quand `isRunning` vaut `true`.
- Toutes les 1 seconde, incrémente `seconds`.
- N'oublie pas de nettoyer (`clearInterval`) quand `isRunning` passe à `false` ou quand le composant est démonté.

Bonus

- Affiche le temps au format `mm:ss` (ex: `01:05` pour 65 secondes).
- Désactive le bouton `Démarrer` quand le timer est déjà en cours.

Exercice 6 – Formulaire de contact avec validation

Exercice 6 – Formulaire de contact avec validation

Objectif: formulaires contrôlés + validation basique.

- Crée un composant `ContactForm`.
- Ajoute trois états pour tes champs :
  - `name`
  - `email`
  - `message`
- Affiche un formulaire avec :
  - un `input` pour le nom,
  - un `input` pour l'email,
  - un `textarea` pour le message,
  - un bouton `Envoyer`.

Validation

- Au `submit`, vérifie :
  - que `name` n'est pas vide,
  - que `email` contient au moins un `@`,
  - que `message` fait au moins 10 caractères.
- Si une règle n'est pas respectée, affiche un petit message d'erreur sous le champ correspondant.
- Si tout est OK, affiche un message de succès : "Message envoyé !" et vide les champs.

Bonus

- Désactive le bouton `Envoyer` tant que au moins une règle n'est pas respectée.

Exercice 7 – Mini "router" avec état

Objectif: simuler une navigation (sans `react-router`).

- Crée un composant `MiniRouter`.
- Crée un état `currentPage` qui peut valoir :
  - `"home"`, `"about"`, `"contact"`.
- Affiche un "menu" avec trois boutons :
  - `Accueil`, `À propos`, `Contact`.
- Quand on clique sur un bouton, mets à jour `currentPage` avec la valeur correspondante.

Affichage conditionnel

- En fonction de `currentPage`, affiche :

- En fonction de `currentPage`, affiche :
  - Pour `"home"` : un titre “Page d’accueil” et un texte.
  - Pour `"about"` : un titre “À propos” et un texte.
  - Pour `"contact"` : un titre “Contact” et un petit formulaire simplifié (par exemple juste un champ email).

Bonus

- Mets la page active en gras ou avec un style différent dans le “menu”